

Universidade Maurício de Nassau
Curso de Ciência da Computação

BuscaPet

Site de adoção e busca de animais perdidos

Documento da Sprint 2

Kaian Guthierry da Silva — 01617843
Nivaldo José de Arruda Filho — 01486768
Marlon Porto Torres — 01611478

Recife — 2026

Sumário

1. Arquitetura do sistema
2. Diagrama de classes
3. Modelo Entidade-Relacionamento (MER)
4. Modelo Relacional
5. Protótipo das telas principais
6. Banco de dados criado
7. Projeto estruturado no GitHub

1. Arquitetura do sistema

1.1 Visão geral

A plataforma é dividida em quatro componentes que se comunicam por HTTP, e não em um monólito. A decisão mais importante da arquitetura é isolar o núcleo de similaridade visual — o componente que atende a disciplina de Tópicos Avançados — do backend web: eles são dois serviços separados, em linguagens diferentes, falando entre si por uma API HTTP interna enxuta.

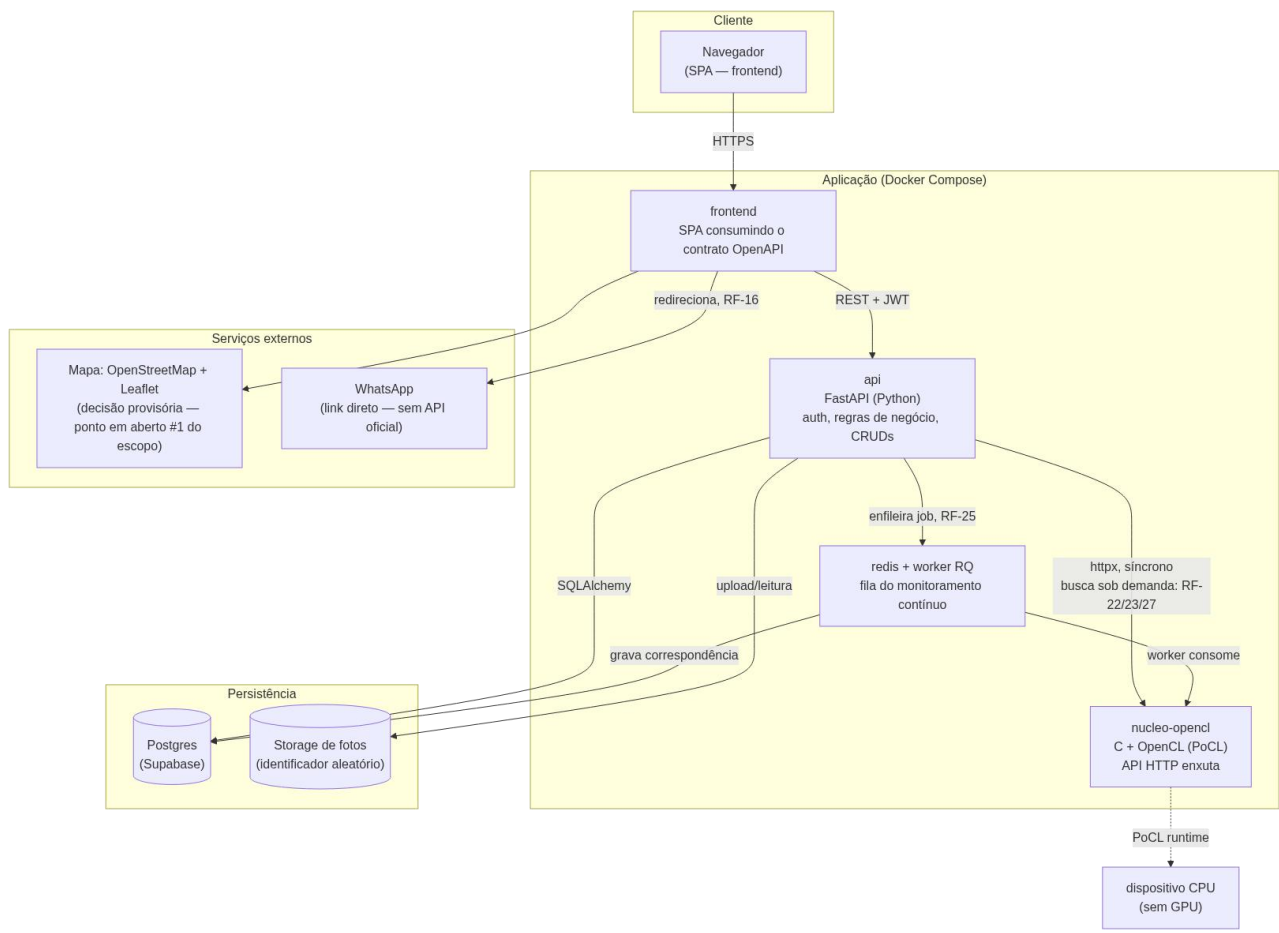


Figura 1 — Diagrama de arquitetura do BuscaPet.

1.2 Componentes e tecnologias

Componente	Tecnologia	Responsabilidade
frontend	SPA (framework definido por Marlon — P3)	Interface das telas descritas no documento de fluxos
api	Python + FastAPI, SQLAlchemy 2.0 + Alembic, JWT	Autenticação e controle de acesso por perfil (RF-01 a RF-03), CRUDs de abrigo/animal/doação, orquestração das chamadas ao núcleo
nucleo-opengl	C (cl.h) + PoCL	Extração de descritores visuais, comparação em massa e benchmark comparativo (RF-21 a RF-23, RF-27, RF-29)
Fila (redis + worker RQ)	Redis + RQ (Python)	Processa o lote de monitoramento contínuo (RF-24, RF-25) sem bloquear requisições da API
Banco de dados	Postgres (Supabase, free tier, PostGIS disponível)	Persistência relacional
Storage de fotos	A definir (Supabase Storage é o candidato natural)	Guarda arquivos com identificador aleatório, sem expor dados do autor

Componente	Tecnologia	Responsabilidade
Mapa	OpenStreetMap + Leaflet	Gratuito e sem cota que ameace o projeto — resolve o ponto em aberto nº1 do escopo

1.3 Por que o núcleo é um serviço separado, e não uma função dentro do backend

- Runtime isolado — o núcleo depende do PoCL, que precisa estar instalado no ambiente de execução. Isso não deve ser um requisito para quem só trabalha no backend web ou no frontend, por isso o Docker Compose isola esse componente em um container próprio.
- Medição isolada — o benchmark comparativo com/sem OpenCL (RF-29, argumento central do projeto para Tópicos Avançados) precisa rodar sem ruído de outras partes da aplicação competindo por CPU.
- Contrato fechado desde a Sprint 1 — a API do núcleo (POST /comparar, POST /extrair-descritor) foi definida como parte do contrato OpenAPI da Sprint 1, permitindo que frontend e backend avançassem com dados simulados antes do núcleo existir de fato (Sprint 3).

1.4 Dois caminhos de chamada ao núcleo

- Sob demanda (RF-22, RF-23, RF-27): o usuário aciona uma busca no wizard “Perdi meu animal” / “Encontrei um animal” → a API chama o nucleo-opencl via httpx de forma síncrona → o resultado volta na mesma requisição.
- Em lote, assíncrono (RF-24, RF-25, RF-26): todo novo registro (Animal ou Avistamento) dispara um job na fila Redis → um worker RQ consome o job, chama o núcleo comparando contra todas as buscas salvas ativas, grava o resultado em correspondências e cria notificações quando há semelhança relevante. Rodar isso fora do ciclo de requisição é o que sustenta a carga N×M crescente sem travar a API.

2. Diagrama de classes

Modelo de domínio do backend. Usuario concentra a autenticação; os três perfis de conta (Tutor, Abrigo, Apoiador) e o Administrador herdam dele. No banco isso é implementado como tabelas separadas em relação 1:1 (table-per-type, ver seção 4), mas no código de domínio a herança é a forma natural de expressar que todo perfil é um usuário.

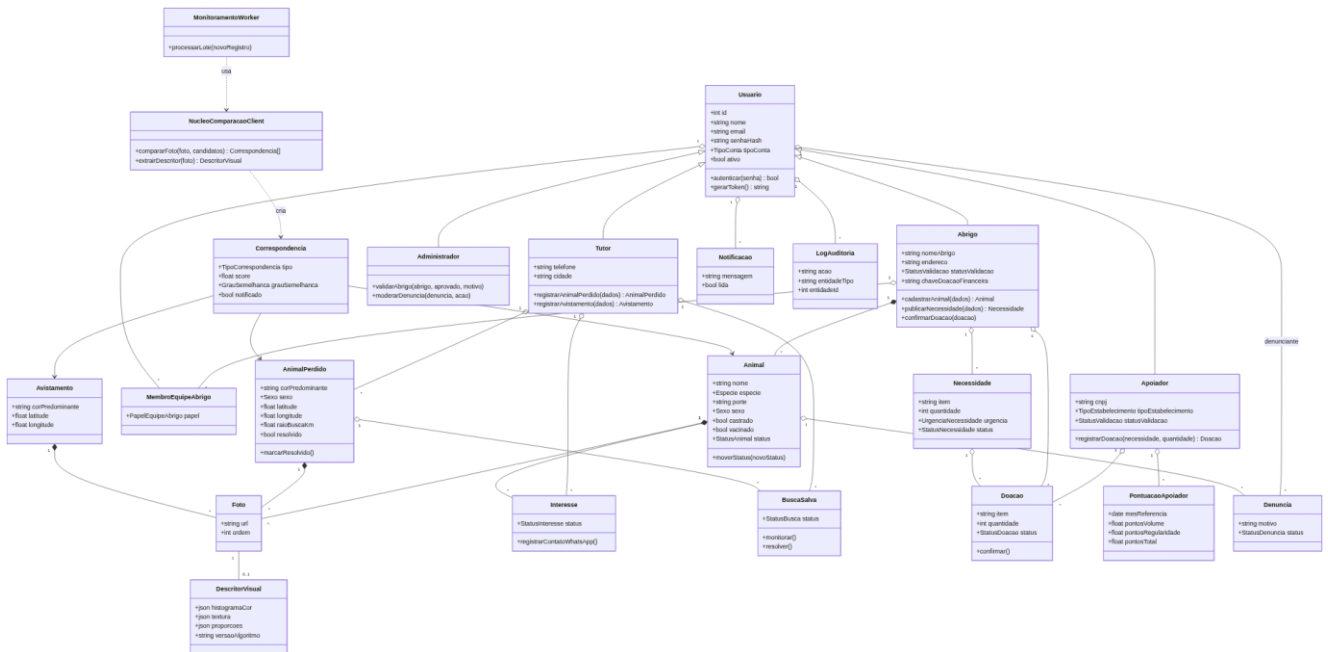


Figura 2 — Diagrama de classes do BuscaPet.

Notas sobre decisões de modelagem

- NucleoComparacaoClient e MonitoramentoWorker são classes de serviço do backend Python, não do núcleo em C. Encapsulam a chamada HTTP (httpx) ao serviço nucleo-openc1, mantendo o resto do domínio (Tutor, Abrigo etc.) sem dependência direta de como o núcleo é implementado — se o núcleo mudar de linguagem no futuro, só essas duas classes precisam mudar.
- Foto é compartilhada por três donos possíveis (Animal, AnimalPerdido, Avistamento) via associação polimórfica — mesmo trade-off explicado nas seções 3 e 4.
- Correspondencia é o registro central do reencontro: toda vez que o núcleo encontra um match relevante — numa busca ativa do tutor, num alerta ao abrigo ou numa busca inversa — uma linha nasce aqui, evitando duplicar a lógica de “o que é um match” em três lugares do código.

3. Modelo Entidade-Relacionamento (MER)

Modelo conceitual completo, cobrindo RF-01 a RF-41. Correspondência 1:1 com docs/schema.sql (modelo relacional) e com api/app/models/ (SQLAlchemy).

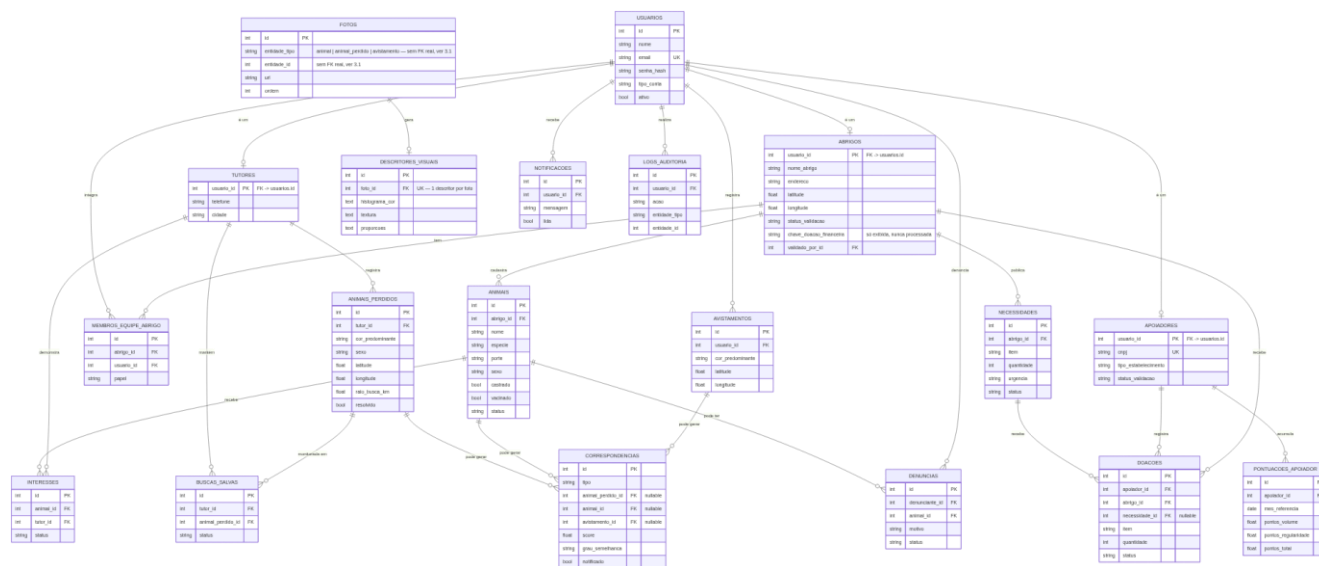


Figura 3 — Modelo Entidade-Relacionamento do BuscaPet.

3.1 Decisões de modelagem que exigem explicação

Contas por tipo (table-per-type), não uma tabela única com colunas opcionais

USUARIOS guarda só o que é comum a todo mundo (autenticação). TUTORES, ABRIGOS e APOIADORES têm PK = FK para usuarios.id. A alternativa descartada — uma tabela usuarios única com todas as colunas de todos os perfis — geraria dezenas de colunas NULL para quem não é abrigo, por exemplo, e dificultaria constraint de obrigatoriedade por perfil.

FOTOS é uma associação polimórfica

Pertence a um ANIMAIS, um ANIMAIS_PERDIDOS ou um AVISTAMENTOS, indicado por entidade_tipo + entidade_id, sem chave estrangeira de banco de fato (não é possível uma FK apontar para “uma dentre três tabelas” em SQL padrão). A alternativa descartada — três tabelas quase idênticas (FOTOS_ANIMAL, FOTOS_PERDIDO, FOTOS_AVISTAMENTO) — foi rejeitada porque a extração de descritores (RF-21) processa foto por foto de forma idêntica não importa a origem, e triplicar a tabela obrigaria triplicar essa lógica ou fazer três JOINS onde um bastaria. A integridade referencial dessa coluna é garantida na camada de aplicação, não pelo banco — um trade-off assumido conscientemente.

CORRESPONDENCIAS *centraliza os três gatilhos de match*

Busca do tutor, alerta ao abrigo e busca inversa, com as três FKs de destino opcionais (só uma é preenchida por linha, a depender de tipo). Isso evita reimplementar “o que é um grau de semelhança” três vezes.

PONTUACOES_APOIADOR é uma tabela de cache/resumo

Não é a fonte de verdade (que são as linhas de DOACOEES confirmadas). Existe para não recalculer o ranking inteiro a cada requisição — é recalculada a cada confirmação de doação, uma linha por apoiador por mês, permitindo os dois períodos do escopo (destaque do mês = uma linha; acumulado do ano = soma das linhas do ano).

4. Modelo Relacional

DDL completo em docs/schema.sql (roda direto no SQL Editor do Supabase), espelhado nos models SQLAlchemy em api/app/models/ e na migration Alembic api/alembic/versions/0001_initial_schema.py — é essa migration que efetivamente cria o banco (ver seção 6).

Tabela	PK	FKs	Observação
usuarios	id	—	Base de autenticação de todos os perfis
tutores	usuario_id	usuario_id → usuarios.id	PK = FK (1:1 com usuarios)
abrigos	usuario_id	usuario_id → usuarios.id; validado_por_id → usuarios.id	status_validacao controla RF-06
membros_equipe_abrigo	id	abrigo_id → abrigos.usuario_id; usuario_id → usuarios.id	UNIQUE(abrigo_id, usuario_id)
apoiadores	usuario_id	usuario_id → usuarios.id	cnpj UNIQUE
animais	id	abrigo_id → abrigos.usuario_id	—
fotos	id	nenhuma FK real (entidade_tipo + entidade_id)	associação polimórfica — ver seção 3
interesses	id	animal_id → animais.id; tutor_id → tutores.usuario_id	RF-16
animais_perdidos	id	tutor_id → tutores.usuario_id	RF-19
avistamentos	id	usuario_id → usuarios.id	RF-20
descritores_visuais	id	foto_id → fotos.id (UNIQUE)	1 descritor por foto — RF-21
buscas_salvas	id	tutor_id → tutores.usuario_id; animal_perdido_id → animais_perdidos.id	RF-24
correspondencias	id	animal_perdido_id, animal_id, avistamento_id (nullable)	RF-23, 25, 26, 27
necessidades	id	abrigo_id → abrigos.usuario_id	RF-30
doacoes	id	apoiador_id, abrigo_id, necessidade_id (nullable)	RF-31, RF-32
pontuacoes_apoiador	id	apoiador_id → apoiadores.usuario_id	UNIQUE(apoiador_id, mes_referencia) — RF-33
notificacoes	id	usuario_id → usuarios.id	RF-36
denuncias	id	denunciante_id, animal_id, resolvido_por_id	RF-41
logs_auditoria	id	usuario_id → usuarios.id	RF-39

Normalização

O modelo está em 3FN: nenhuma coluna não-chave depende de outra coluna não-chave (ex.: pontuacoes_apoiador é uma tabela de cache separada, não colunas de “pontos” espalhadas em apoiadores, para não misturar um dado derivado/recalculável com o dado de cadastro). A única quebra deliberada de integridade referencial estrita é a de fotos.entidade_id, documentada na seção 3.1.

Índices além das chaves

- ix_fotos_entidade em fotos (entidade_tipo, entidade_id) — toda leitura de fotos de um animal/perdido/avistamento filtra por essas duas colunas.

- Índices de geolocalização (busca por raio no mapa, RF-07 a RF-11) ficam para quando o volume justificar — o Supabase já tem PostGIS disponível se for necessário migrar latitude/longitude para um tipo geography. Ponto em aberto, não bloqueia a Sprint 2.

5. Protótipo das telas principais

Protótipo em alta-fidelidade (Figma):

<https://www.figma.com/design/hRa8GLWzc4vvVToGGQ4Zxe/Untitled?node-id=0-1>

Wireframe original (base para a versão no Figma) também salvo em sprints/sprint-2/prototipo.html — abrir localmente no navegador funciona sem servidor. As sete telas principais estão reproduzidas abaixo; cada uma traz anotações marcadas DOC (já especificado no escopo), DOC parcial (existe mas incompleto) ou IDEIA (decisão da equipe, ainda não formalizada) — a mesma legenda usada no documento de descrição de páginas e fluxos, para manter rastreabilidade entre o que foi decidido e de onde veio a decisão.

Por que wireframe em vez de alta-fidelidade agora

Nesta Sprint o objetivo é validar fluxo e organização de informação, não identidade visual — decidir cor, tipografia e ilustração definitivas antes de fechar os fluxos (principalmente os pontos em aberto do reencontro e do perfil administrador, ainda os menos amadurecidos) seria retrabalho garantido. O wireframe usa caixas rotuladas e hachurado no lugar de fotos reais, mantendo o foco em: o que existe em cada tela, onde o usuário clica, e o que abre em seguida.

5.1 Autenticação

Requisitos: RF-02, escopo seção 2.1. Login único: o sistema identifica o perfil pelo e-mail cadastrado, sem botões separados por tipo de conta.

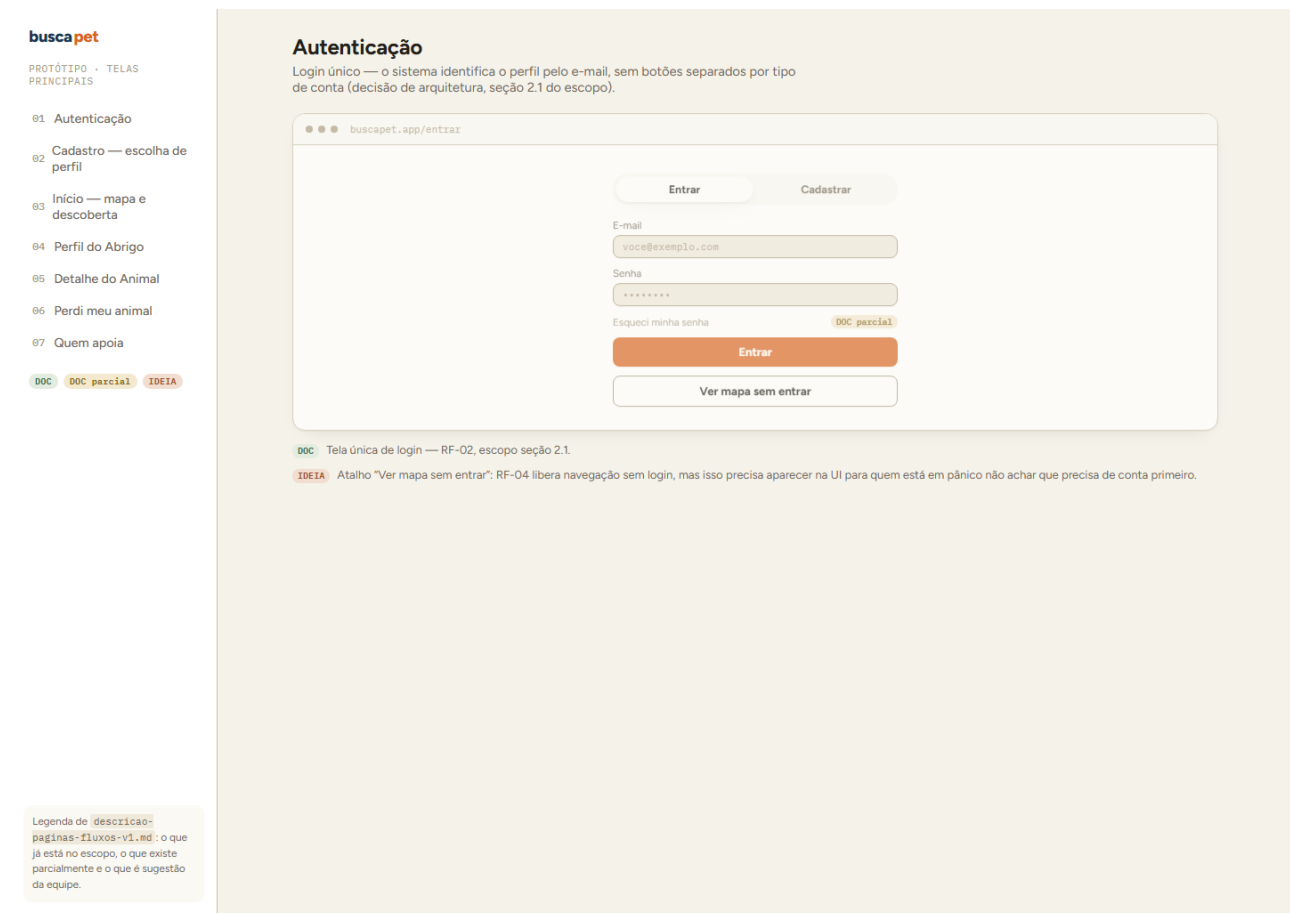


Figura 4 — Tela "Autenticação" do protótipo.

5.2 Cadastro — escolha de perfil

Requisitos: RF-01. Escolha entre tutor, abrigo/protetor ou apoiador no momento do cadastro.

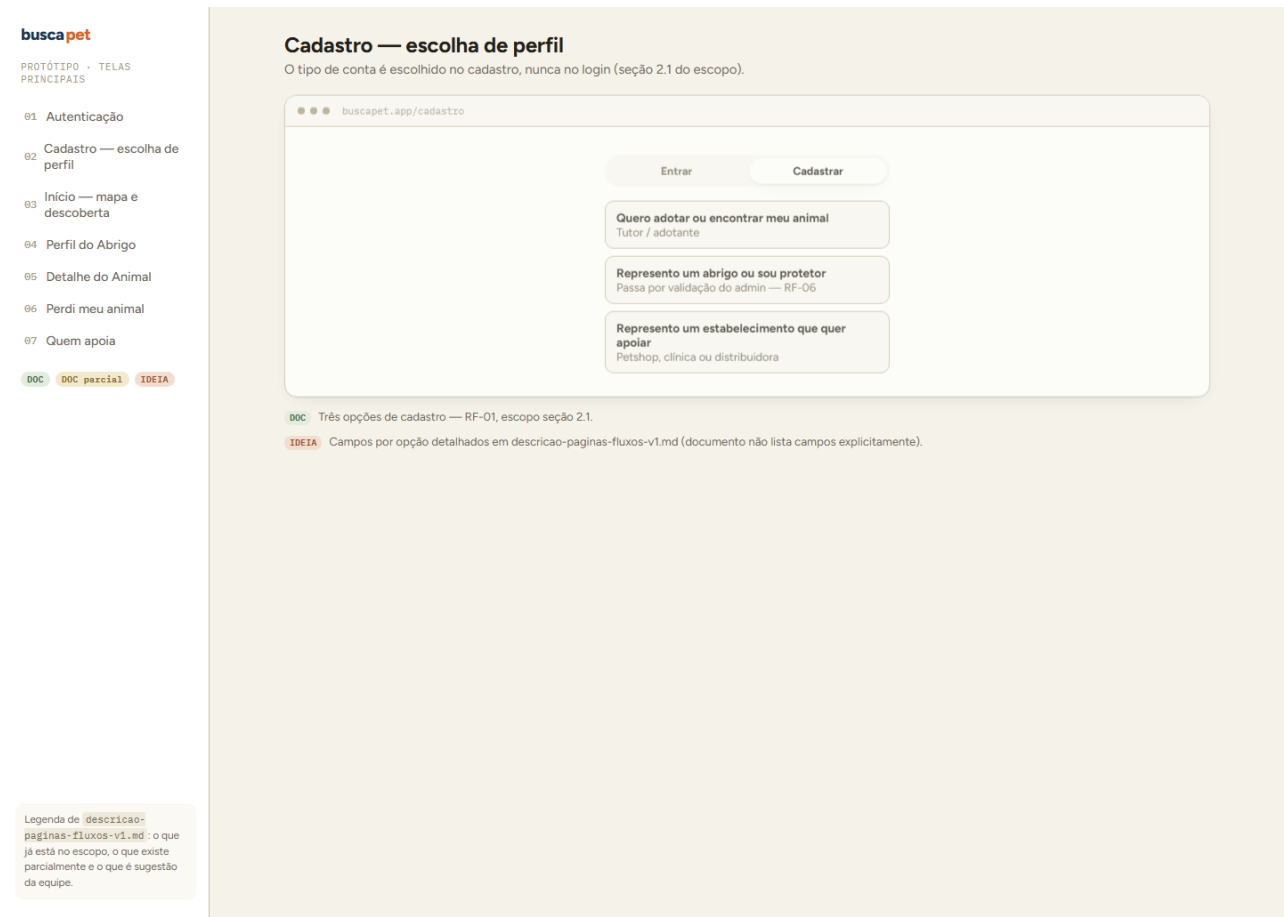


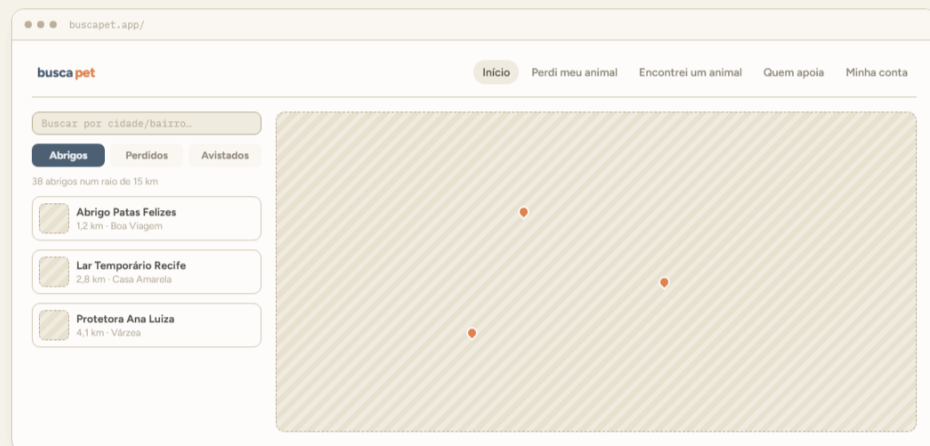
Figura 5 — Tela "Cadastro — escolha de perfil" do protótipo.

5.3 Início — mapa e descoberta

Requisitos: RF-07, RF-08, RF-09. Mapa central com lista lateral sincronizada, abas por camada (abrigo, perdidos, avistados).

Início — mapa e descoberta

Mapa central com lista lateral sincronizada, abas por camada (RF-07 a RF-09).



DOC Mapa + lista sincronizada, abas Abrigos/Perdidos/Avistados — RF-07 a RF-09.

IDEIA Campo de busca por cidade/bairro e contador "38 abrigos num raio de 15 km", conectando com RF-11.

Figura 6 — Tela "Início — mapa e descoberta" do protótipo.

5.4 Perfil do Abrigo

Requisitos: RF-12, RF-13, RF-14. Informações do abrigo, catálogo de animais em grade e filtros de busca.

- buscapet**
- PROTÓTIPO • TELAS
PRINCIPAIS
- 01 Autenticação
 - 02 Cadastro — escolha de perfil
 - 03 Início — mapa e descoberta
 - 04 Perfil do Abrigo
 - 05 Detalhe do Animal
 - 06 Perdi meu animal
 - 07 Quem apoia
- DOC DOC parcial IDEIA

Legenda de descrição-
páginas-fluxos-vi.md: o que
já está no escopo, o que existe
parcialmente e o que é sugestão
da equipe.

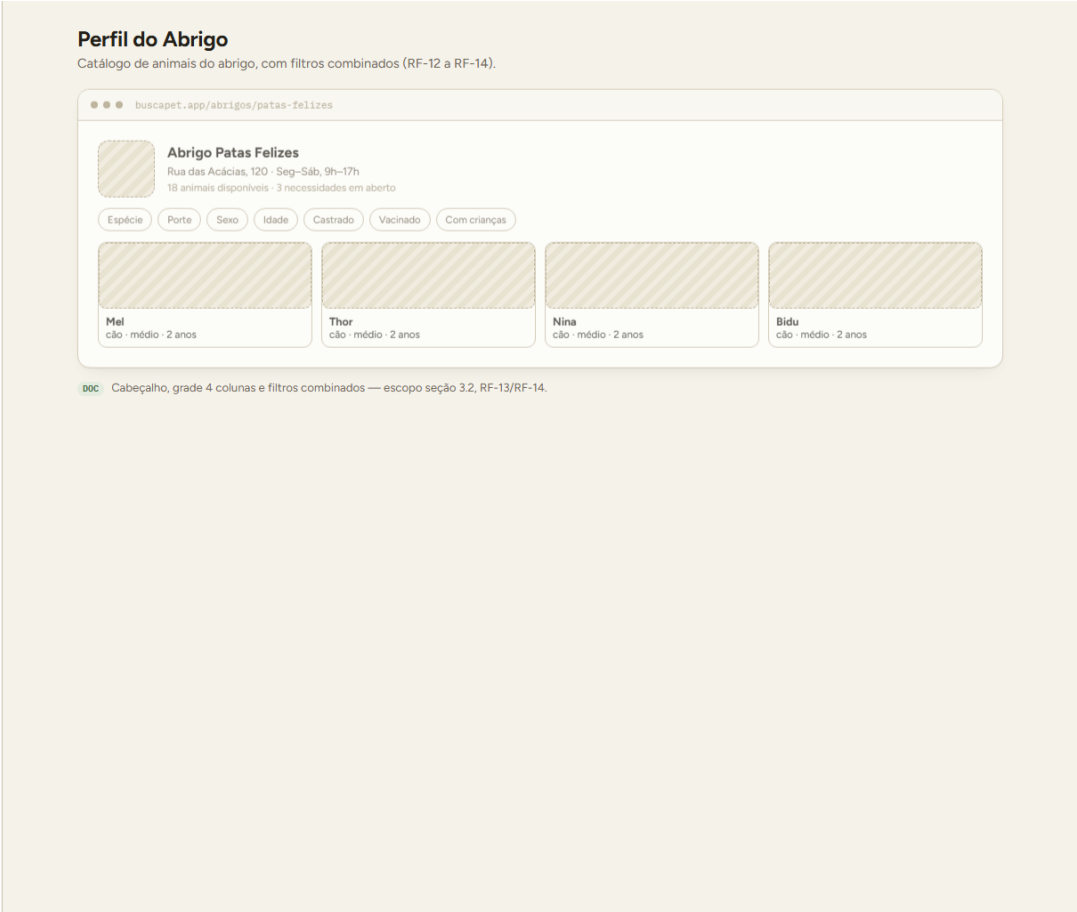


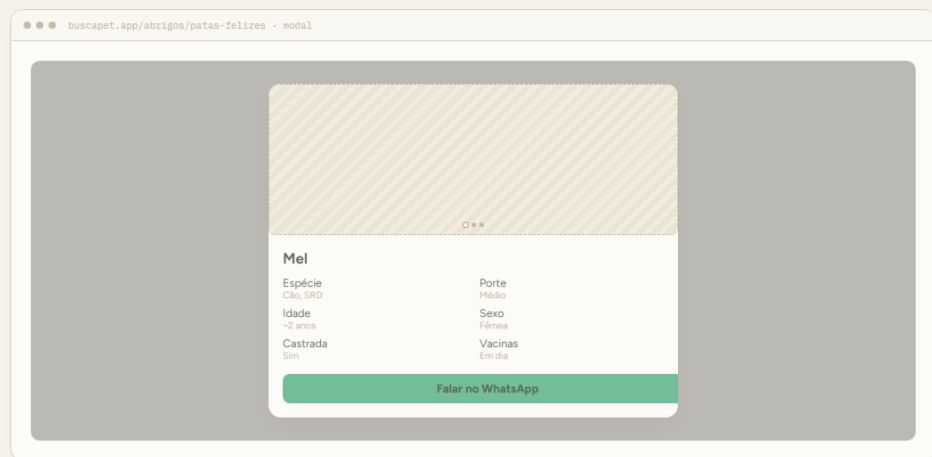
Figura 7 — Tela "Perfil do Abrigo" do protótipo.

5.5 Detalhe do Animal (modal)

Requisitos: RF-15, RF-16. Galeria de fotos, descrição completa e contato por WhatsApp com registro do interesse.

Detalhe do Animal

Sobreposição com galeria e ficha completa — o clique no WhatsApp registra interesse antes de redirecionar (RF-16).



DOC Ficha e galeria navegável — escopo seção 3.3.

DOC Registro de interesse antes do redirecionamento — RF-16: "um clique, um registro".

Figura 8 — Tela "Detalhe do Animal (modal)" do protótipo.

5.6 Perdi meu animal (wizard)

Requisitos: RF-19, RF-21 a RF-23. Etapas do wizard de registro de animal perdido e busca por similaridade visual.

- 01 Autenticação
 - 02 Cadastro — escolha de perfil
 - 03 Início — mapa e descoberta
 - 04 Perfil do Abrigo
 - 05 Detalhe do Animal
 - 06 Perdi meu animal
 - 07 Quem apoia
- DOC DOC parcial IDEIA

Legenda de descrição-
páginas-fluxos-vi.md: o que
já está no escopo, o que existe
parcialmente e o que é sugestão
da equipe.

Perdi meu animal

Wizard de 6 etapas com progresso visível (escopo seção 4.2). Alternar abaixo entre a Etapa 1 e a Etapa 5.

buscapet.app/perdi-meu-animal

Etapa 1 — Fotos Etapa 5 — Resultados

1 Fotos 2 Características 3 Local 4 Processando 5 Resultados 6 Salvar

Arraste de 1 a 5 fotos, ou toque para escolher
corpo inteiro · boa luz · ângulos diferentes

The wizard interface is titled 'Perdi meu animal' and is a 6-step process. The progress bar shows steps 1 through 6: 'Fotos', 'Características', 'Local', 'Processando', 'Resultados', and 'Salvar'. Step 1 is currently active. Below the progress bar is a dashed box containing the instruction 'Arraste de 1 a 5 fotos, ou toque para escolher corpo inteiro · boa luz · ângulos diferentes'. Below this instruction are three large, empty rectangular boxes for photo uploads, each with a dashed border and diagonal hatching.

DOC Wizard de 6 etapas com progresso visível — escopo seção 4.2.

DOC Grau de semelhança qualitativo (alta/média/baixa), nunca porcentagem crua ou confirmação de match — escopo seção 4.3.

Figura 9 — Tela "Perdi meu animal (wizard)" do protótipo.

5.7 Quem apoia

Requisitos: RF-30, RF-33, RF-35. Mural de necessidades dos abrigos e ranking de apoiadores.

- 01 Autenticação
- 02 Cadastro — escolha de perfil
- 03 Início — mapa e descoberta
- 04 Perfil do Abrigo
- 05 Detalhe do Animal
- 06 Perdi meu animal
- 07 Quem apoia

DOC DOC parcial IDEIA

Legenda de descrição-
páginas-fluxos-vi.md: o que
já está no escopo, o que existe
parcialmente e o que é sugestão
da equipe.

Quem apoia

Ranking por categoria + mural de necessidades — combina reconhecimento com conversão (escopo seção 5.3).

buscapet.app/quem-apoia

Petshop Clínica Distribuidora Pessoa física

Destaque do mês · Ração & Cia
"Com a doação mensal conseguimos manter os 40 animais alimentados mesmo nos meses mais difíceis." — Abrigo Patas Felizes

1

Ração & Cia 6 meses seguidos

selo ativo

2

Petshop Amigo Fiel 4 meses seguidos

selo ativo

3

VetCare Clínica 2 meses seguidos

selo ativo

Ração adulta · 20 sacos

alta

Areia sanitária · 40 un.

média

Vermifugo · 15 un.

baixa

Quero apoiar

DOC Categorias, pontuação por regularidade e mural combinado — escopo seção 5.2/5.3.

IDEIA Selo "ativo" no cartão do ranking — reforça a contrapartida ao apoiador (seção 5.4).

Figura 10 — Tela "Quem apoia" do protótipo.

Próximo passo (fora da Sprint 2)

Quando os pontos em aberto de fluxo forem fechados pela equipe, refinar estas telas em alta-fidelidade — Figma ou outra ferramenta — já com identidade visual do BuscaPet.

6. Banco de dados criado

A estrutura inicial está implementada e pronta para ser aplicada — não com dados fictícios prontos ainda (isso é trabalho de “gerador de dados de teste”, já previsto no plano de sprints), mas o schema completo do modelo relacional da seção 4 já existe em duas formas equivalentes:

- Migration Alembic (`api/alembic/versions/0001_initial_schema.py`) — a forma “de verdade” de criar o banco, com histórico versionado (`cd api && pip install -r requirements.txt && alembic upgrade head`).
- DDL direto (`docs/schema.sql`) — mesma estrutura, para quem preferir colar direto no SQL Editor do Supabase sem passar por Python/Alembic.

Onde o banco está hospedado

Decisão de DevOps: Supabase free tier (Postgres com PostGIS disponível). O projeto Supabase free pausa após 1 semana de inatividade — reativar pelo painel antes de cada Sprint/apresentação é parte do checklist de entrega, não um imprevisto. A migration inicial já foi aplicada com sucesso no projeto do Supabase criado para a equipe: 19 tabelas de domínio mais a tabela de controle `alembic_version` foram confirmadas.

O que falta (não é escopo da Sprint 2)

- Popular com dados de teste (abrigos, animais e fotos fictícias).
- Índices de performance além dos mínimos de integridade — só quando o volume justificar.
- A extração de descritores visuais em si (RF-21) não roda ainda — a coluna e a tabela `descritores_visuais` existem, o algoritmo é trabalho do núcleo OpenCL na Sprint 3.

7. Projeto estruturado no GitHub

Organização: <https://github.com/Equipe-BuscaPet>

O código do sistema (backend, núcleo OpenCL e frontend) vive no repositório BuscaPet-App; a documentação de escopo, sprints e diagramas vive em repositório separado (BuscaPet-Docs), para manter o histórico de decisões independente do histórico de código.

Estrutura entregue nesta Sprint (BuscaPet-App)

- `api/` — backend FastAPI: models, config, esqueleto de rotas, testes, Alembic
- `nucleo-opencl/` — serviço C/OpenCL: contrato de API definido, Dockerfile com PoCL, implementação na Sprint 3
- `frontend/` — a estruturar por Marlon (P3 — frontend/UX)
- `docker-compose.yml` — orquestra os 5 serviços locais (frontend, api, nucleo-opencl, db, redis)
- `.github/workflows/ci.yml` — lint (ruff) e testes (pytest) a cada Pull Request
- `CLAUDE.md` — contexto do projeto para quem usa IA no desenvolvimento

Checklist de entrega

Item	Status
Estrutura inicial de pastas	Concluído
README.md com descrição do projeto	Concluído
Modelos de banco, migration inicial e DDL versionados	Concluído
Workflow de CI configurado (lint + testes a cada Pull Request)	Concluído
Primeiro commit de cada integrante	Concluído
Repositório criado na organização Equipe-BuscaPet	Concluído
Documentação técnica revisada e sem divergências entre diagramas	Concluído